

FloodEye

Real-time Flood Alert & Water Level Monitoring System

"Every Drop Monitored, Every Life Protected."

Project Domain: IoT, Machine Learning & Web Dashboard

Technology Innovation Hub (TIH), IIT Guwahati

Team Members



Ashish Kumar Mahato

Mentor



Moharnab Gogoi

IoT Backend & Cloud
Integration



Aryyaman Bora

Frontend & UI
Engineering



Mayuree Khanikar

Documentation &
Research



Indrani Gogoi

Documentation &
Research

Table of Contents

01	Project Overview & Motivation
02	Problem Statement & Objectives
03	System Architecture & Workflow
04	Technology Stack & Hardware Components
05	UI Screens & Features
06	Machine Learning Implementation & Pipeline
07	Experimental Results & Limitations
08	Conclusion & Future Scope

The Problem vs. The Solution

Vulnerability

Millions affected by
Assam floods
annually.

Current System

Historically reactive.
Narrow evacuation
windows.

FloodEye Solution

IoT + ML edge
system for proactive
warning.

Problem Statement & Existing Challenges

Manual Collection

Delays & data gaps

No Prediction

Only shows current state

High Costs

Expensive server dependency

Alert Fatigue

Continuous spam alerts

Vision & Mission

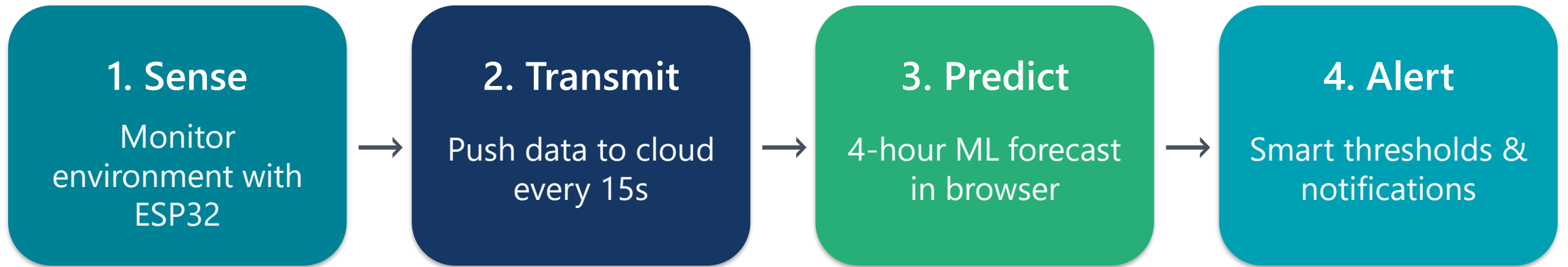
"Proactive, AI-driven IoT systems protecting vulnerable communities from devastating floods."

Protect Lives

Democratize Tech

Deploy Research

Project Objectives



Key System Features

AI Prediction

4-hour browser forecasting

PWA

Offline caching & installable

Live Map

Real-time spatial overlays

Telemetry

5 channels with trend arrows

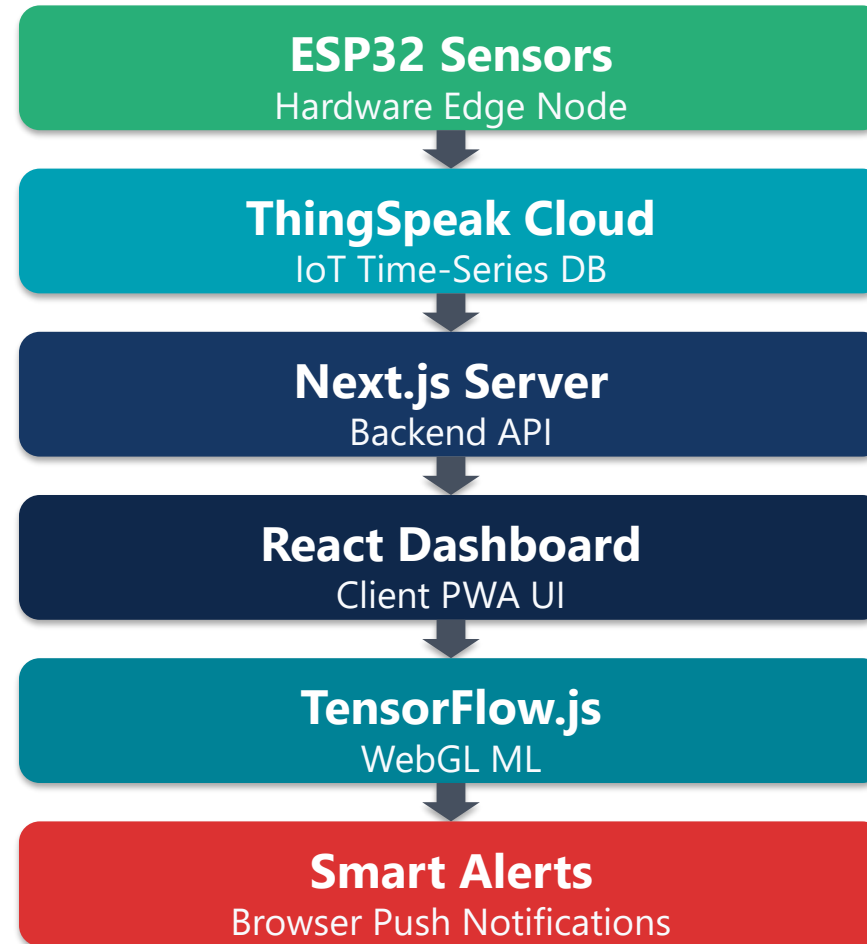
Analytics

Interactive time-series charts

Smart Alerts

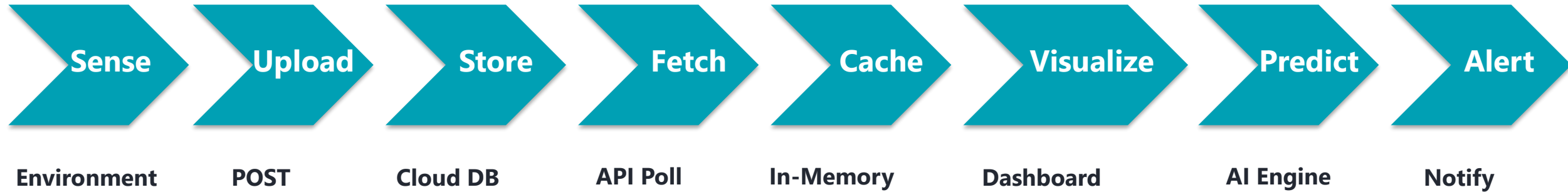
Cooldowns & push notifications

End-to-End System Architecture



Functional Workflow

Functional Process Workflow



Technology Stack: Frontend

Frontend & Cloud Technologies

React.js

Component Web UI

Next.js

Server-Side Rendering

Tailwind

Utility Styling

Vercel

Serverless Host

ThingSpeak

IoT Cloud DB

PWA

Offline App

Technology Stack: Backend & ML

Backend & Machine Learning

Node.js

Async Backend

REST APIs

Stateless Comms

TensorFlow

Keras Training

TensorFlow.js

WebGL Inference

Scikit-Learn

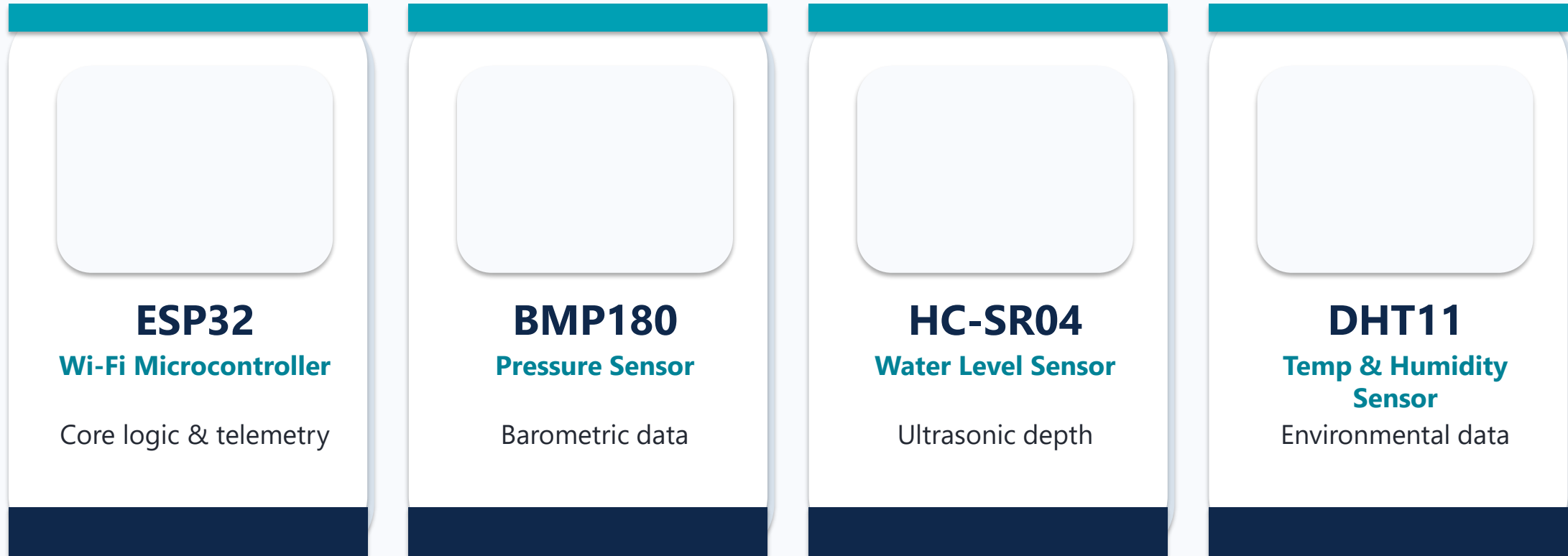
Preprocessing

Next Routes

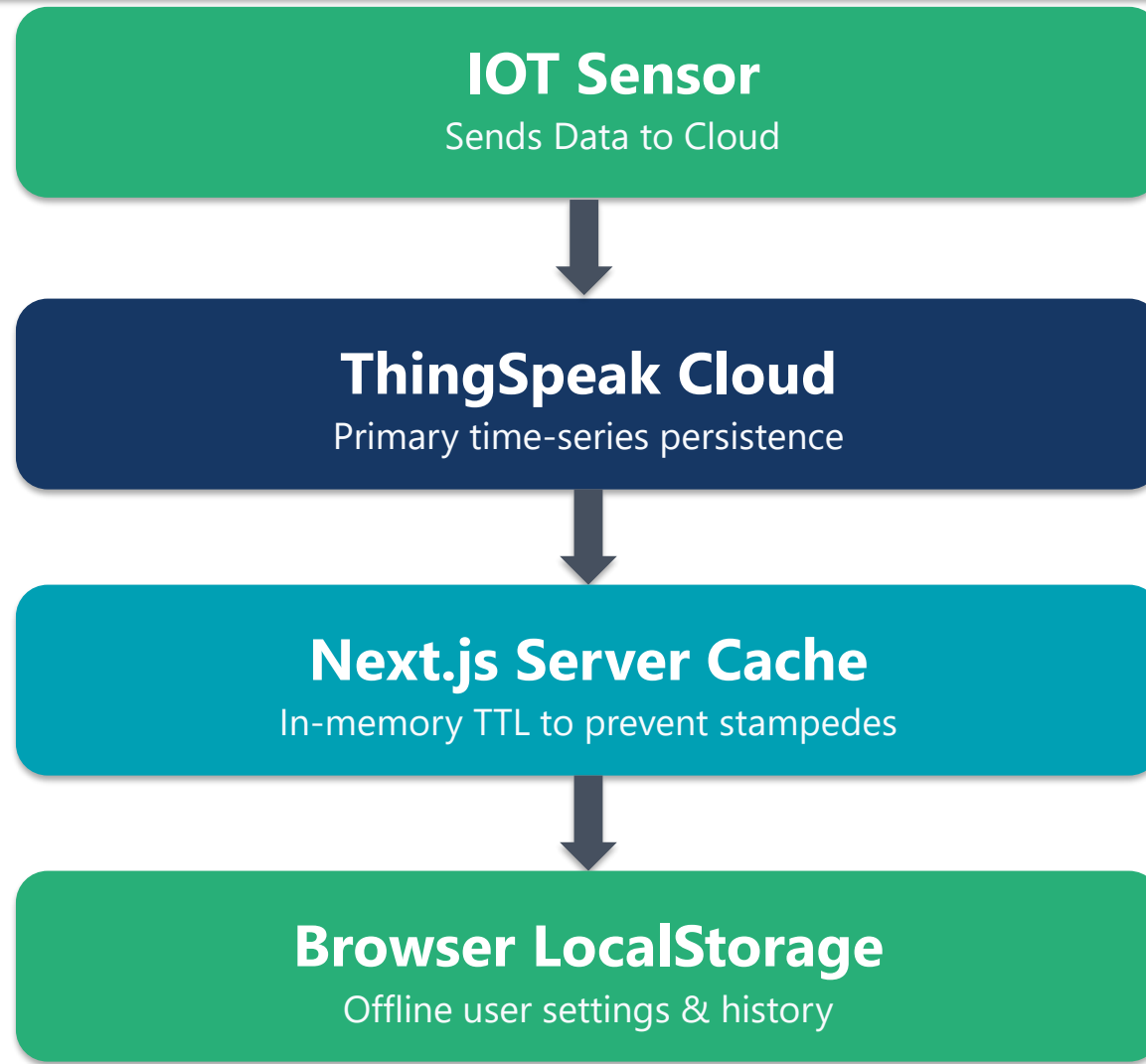
API Coalescing

Hardware Components & Edge Workflow

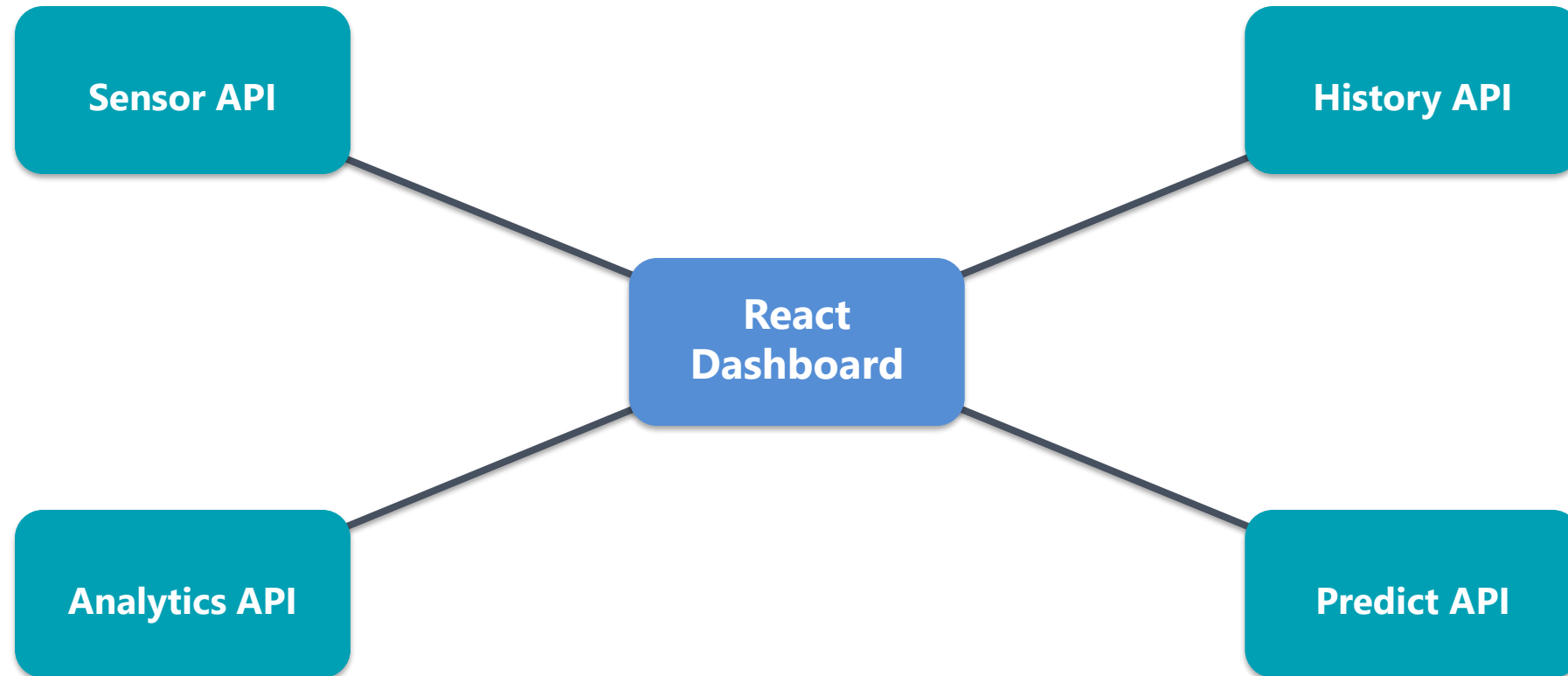
Hardware Components



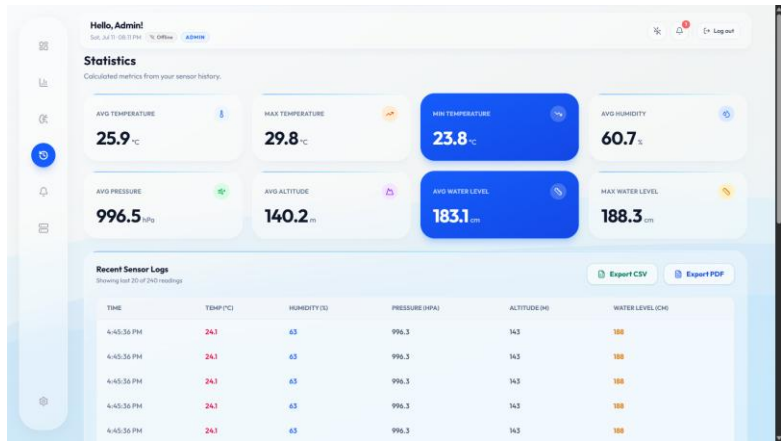
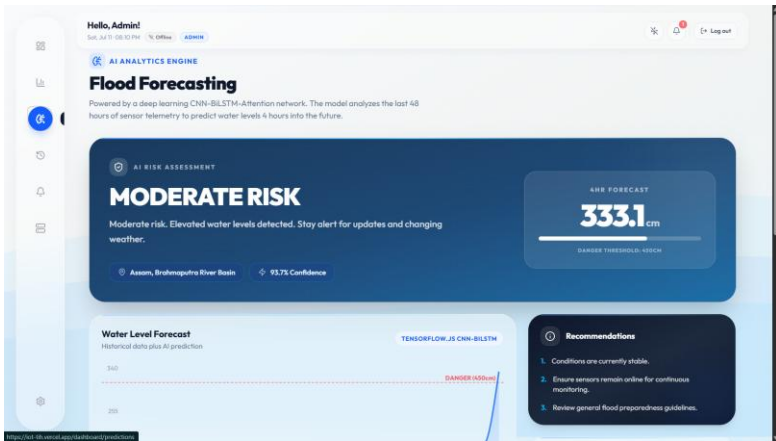
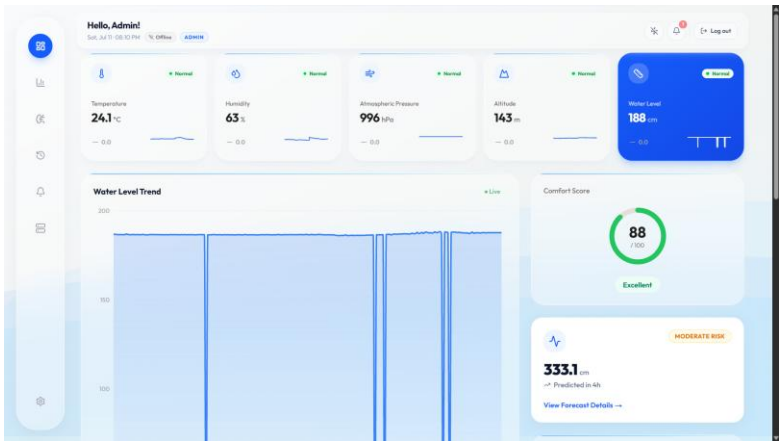
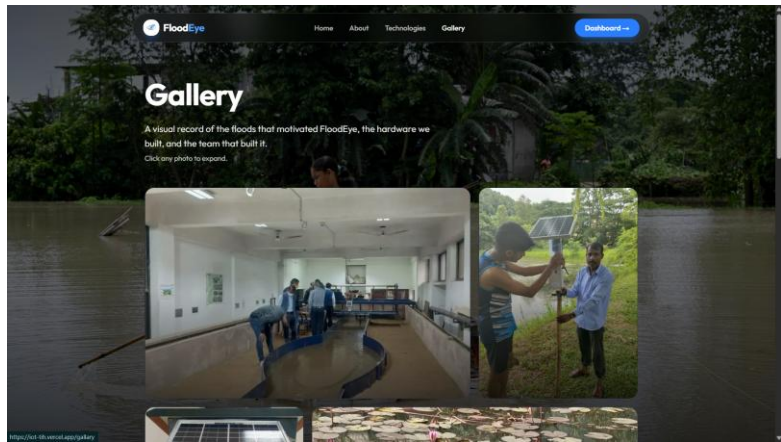
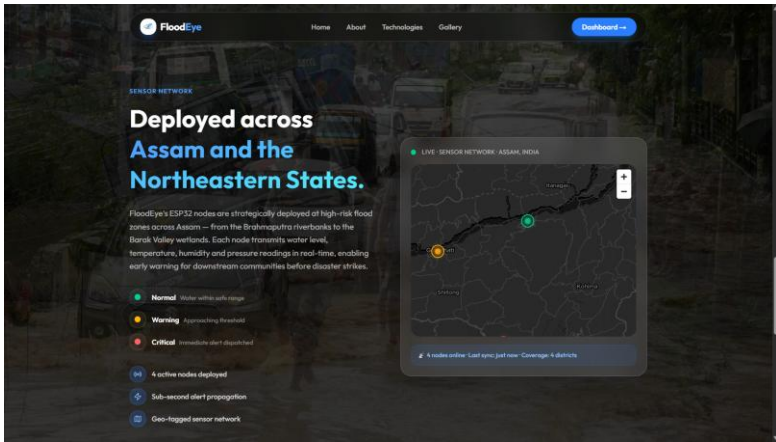
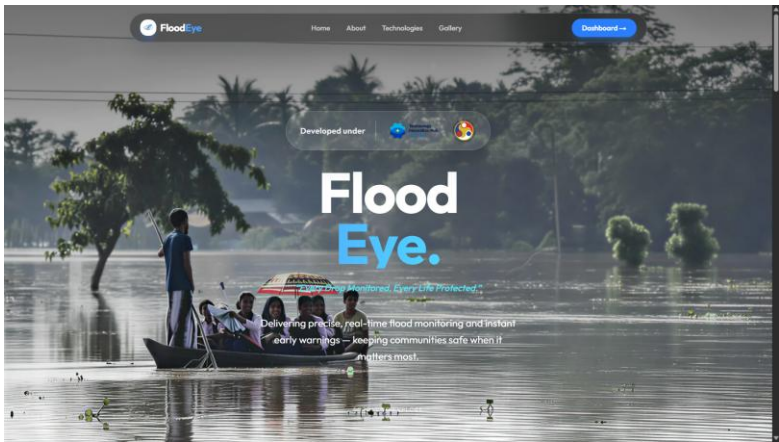
Database Design & Data Storage



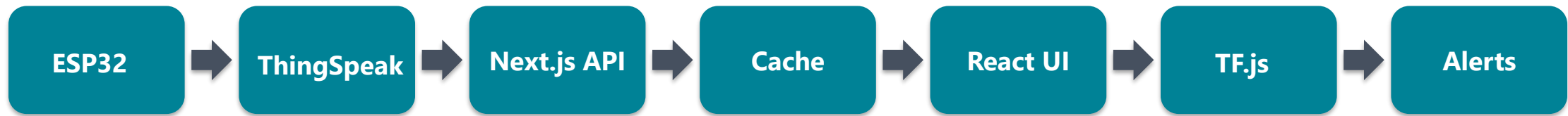
API & Communication Flow



UI Screens



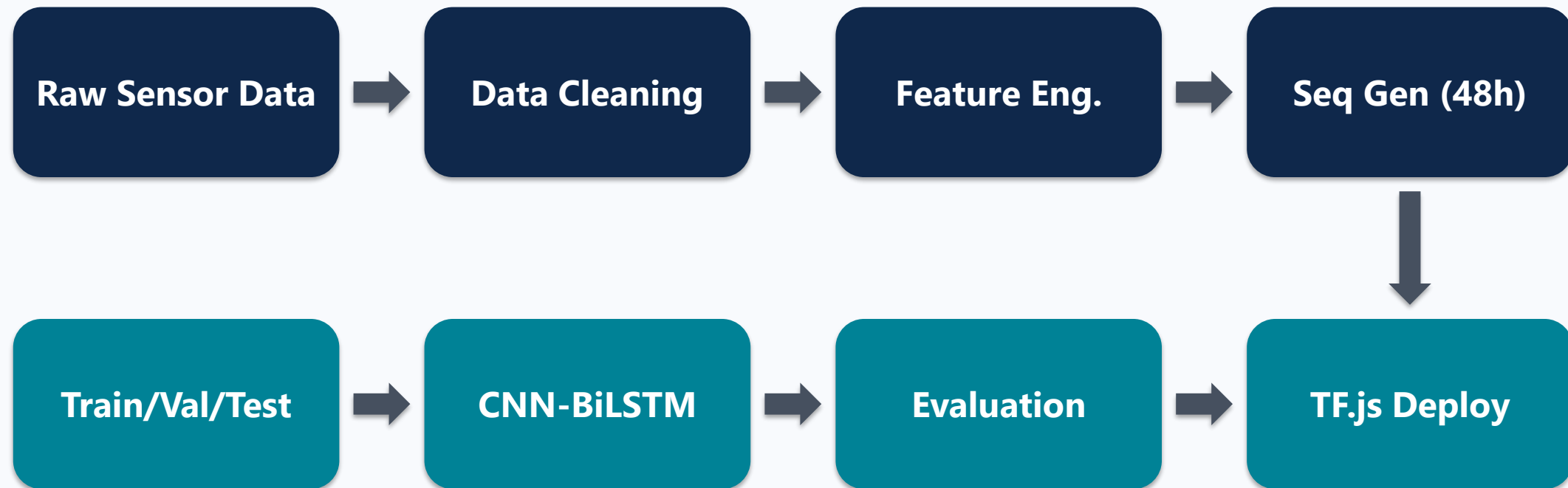
Data Flow & Working Process



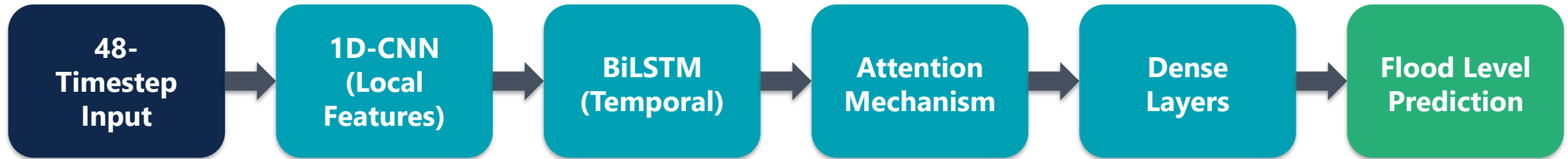
- ESP32 Data Collection:** The ESP32 continuously collects real-time sensor readings and transmits them to the cloud.
- ThingSpeak Cloud:** Sensor data is securely stored and updated on the ThingSpeak platform for remote access.
- Next.js API:** The backend retrieves the latest sensor data, processes requests, and serves it to the application.
- Cache Layer:** Frequently accessed data is temporarily stored to reduce latency and improve system performance.
- React User Interface:** The dashboard displays live sensor readings, flood levels, and system status in an interactive interface.
- TensorFlow.js Inference:** The deployed machine learning model performs real-time flood level prediction directly in the browser.
- Alert Generation:** Based on the predicted flood level, the system generates timely alerts to notify users of potential flood risks.

Machine Learning Methodology

This workflow transforms raw sensor data into accurate flood predictions through preprocessing, feature engineering, sequence generation, **CNN-BiLSTM** model training, evaluation, and deployment using **TensorFlow.js** for real-time web-based inference.

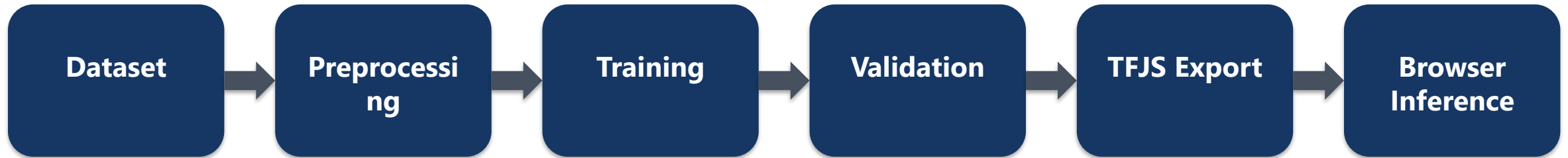


ML Architecture: 1D-CNN + BiLSTM + Attention



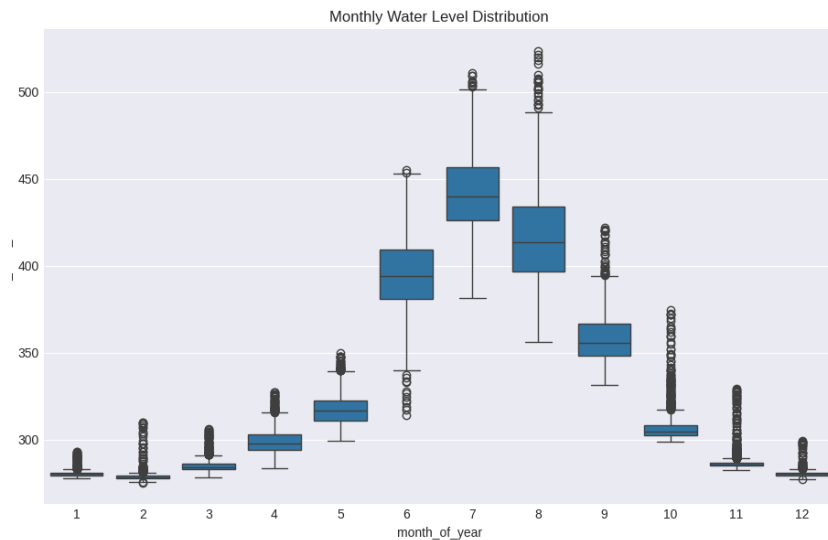
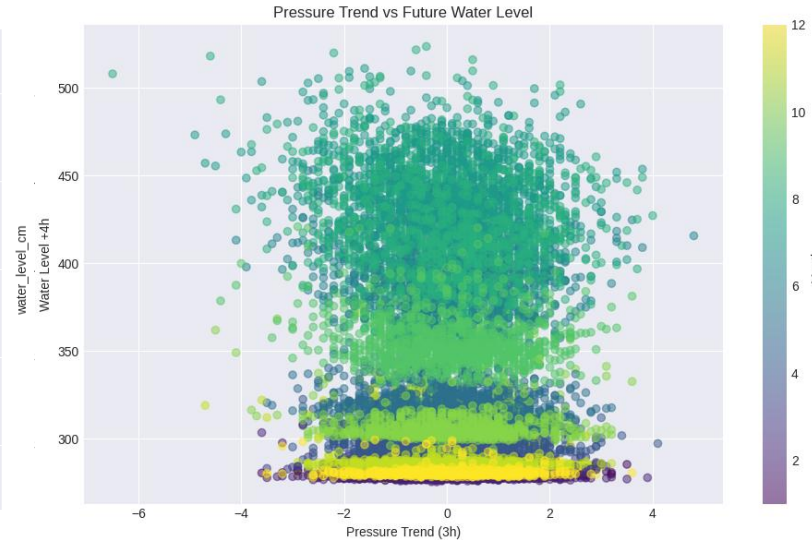
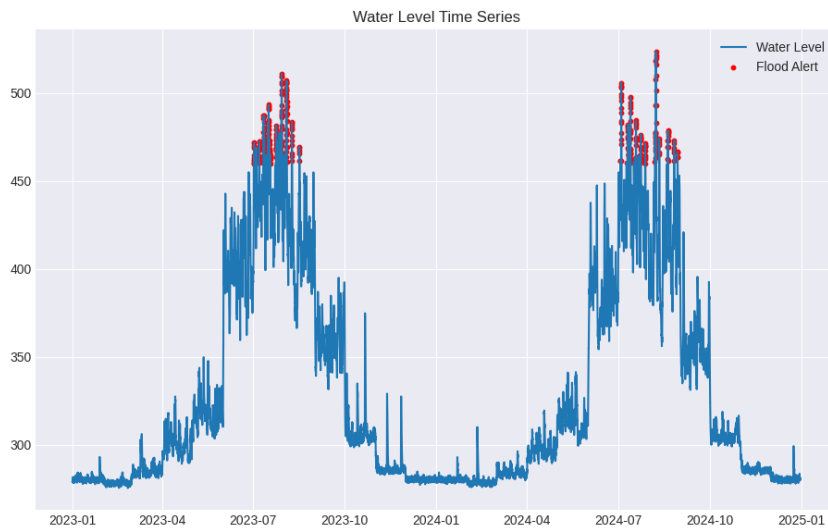
- **48-Timestep Input:** Uses the previous 48 hours of sensor readings as input to capture recent flood trends.
- **1D-CNN Layer:** Extracts important local patterns and features from the sequential sensor data.
- **BiLSTM Layer:** Learns temporal dependencies by analyzing the sequence in both forward and backward directions.
- **Attention Mechanism:** Focuses on the most relevant time steps, improving prediction accuracy during critical events.
- **Dense Layers:** Combine the extracted features to generate the final prediction. **Flood Level Prediction:** Produces accurate real-time flood level forecasts for early warning applications.

Model Training & Browser Inference



- Dataset Preparation:** Historical sensor data is collected and organized for model development.
- Preprocessing:** Data is cleaned, normalized, and converted into fixed 48-hour input sequences.
- Model Training:** The CNN-BiLSTM-Attention model is trained to learn spatial and temporal flood patterns.
- Validation:** Model performance is evaluated using unseen validation data to ensure reliability.
- TensorFlow.js Export:** The trained model is converted into TensorFlow.js format for web deployment.
- Browser Inference:** Predictions are performed directly in the browser, enabling fast, real-time flood monitoring without requiring a server.

Exploratory Data Analysis



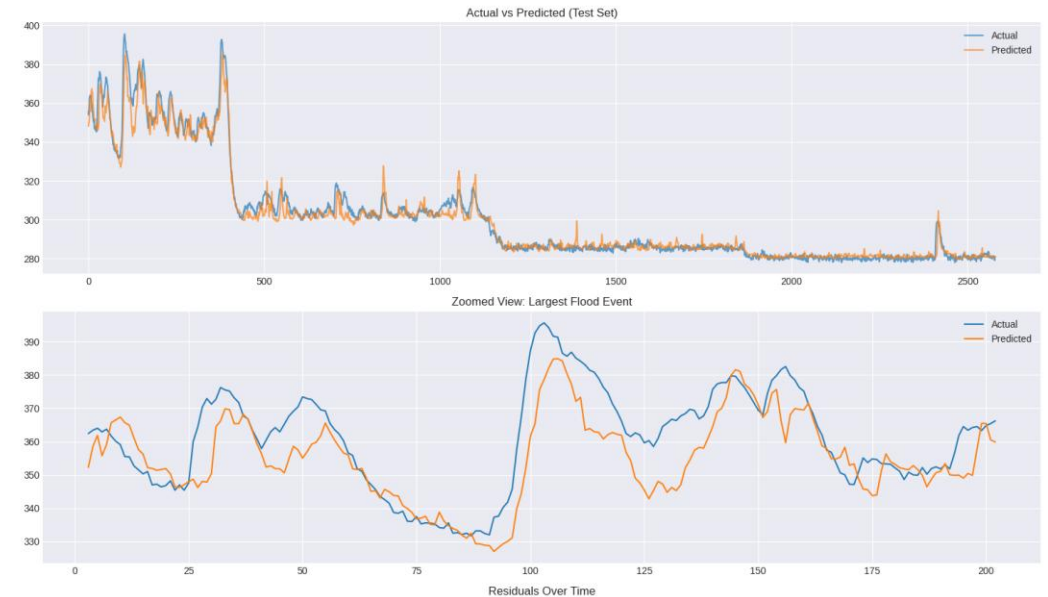
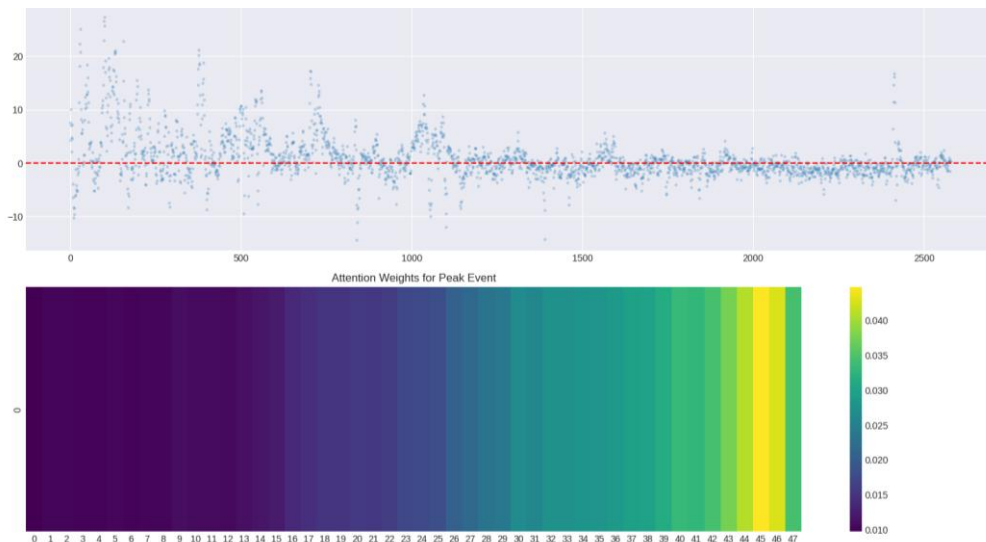
Correlation with Target

water_level_plus_4h	1
water_level_cm	0.99
humidity_percent	0.56
temperature_c	0.48
flood_alert_now	0.37
month_of_year	0.18
rate_of_change_cm_per_hr	0.076
humidity_trend_3h	0.016
hour_of_day	0.00062
pressure_trend_3h	-0.029
pressure_hpa	-0.87
distance_cm	-0.99

- **Seasonal Water Level Trends:** Water levels follow a clear seasonal pattern, peaking during the monsoon months (July–September) and remaining low during the dry season.
- **Flood Event Detection:** Extreme water level peaks correspond to flood alerts, validating the threshold-based labelling used for model development.
- **Monthly Distribution:** Box plot analysis shows higher median water levels and greater variability during the rainy season, indicating elevated flood risk.
- **Feature Relationships:** Current water level is strongly correlated with the 4-hour future water level, while atmospheric pressure shows a strong negative correlation.

Experimental Results

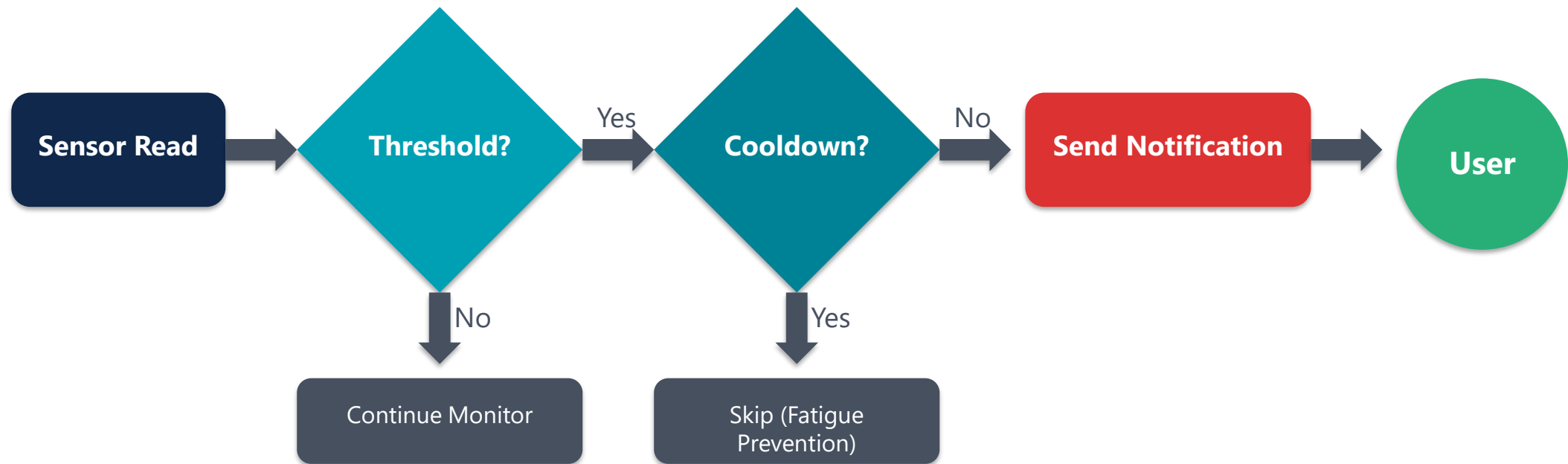
- **Actual vs Predicted Results:** The CNN-BiLSTM model closely follows the actual sensor readings across the test dataset, demonstrating strong prediction accuracy and good trend capture.
- **Peak Event Analysis:** A zoomed-in view of the largest flood event shows that the model successfully captures the rise and fall of water levels, with only slight underestimation at the highest peak.



- **Residual Error Distribution:** The residual plot indicates that most prediction errors are centred around zero, suggesting low bias and stable model performance over time. Larger errors occur only during sudden fluctuations.
- **Attention Weight Visualization:** The attention heatmap highlights that the model assigns greater importance to the most recent time steps within the 48-hour input sequence, enabling more accurate forecasting of upcoming flood events.

Alert System Performance

Smart Alert Decision Engine



Practical Applications & Advantages

Practical Applications

Community Flood Monitoring

Localized early warning networks.

Agricultural Protection

Providing farmers vital prep-time.

Municipal Disaster Management

City-wide dashboard visibility.

Low-Cost IoT Deployment

Under €15 per scalable node.

Challenges & Limitations

Technical Challenges

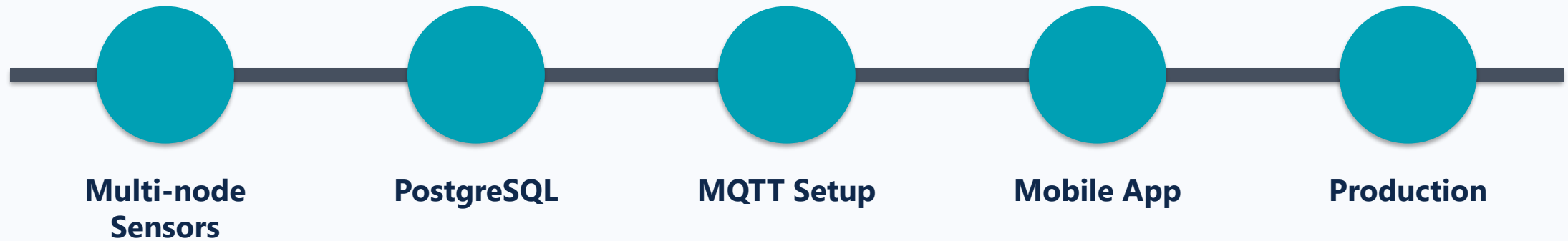
- Keras 3 → TF.js Topology mismatch
- Custom Attention Layer Rewrite in TS
- 48hr Initialization Data Padding
- Preventing Cache Stampedes

Current Limitations

- Single Node Architecture
- No Persistent User Database
- Simulated Confidence Bounds
- ThingSpeak Free-Tier Constraints

Future Enhancements

Future Roadmap



Conclusion

Project Achievements

✓ Real-Time Monitoring

15-second latency telemetry

✓ AI Flood Prediction

4-hour advanced warning window

✓ Smart Alerts

Browser-level push notifications

✓ Low-Cost IoT Solution

Under €15 per node

Real-Time Flood Prediction Network

Questions & Answers

